



INSTITUTO FEDERAL
Rio Grande do Sul

Banco de Dados II

Técnicas de Recuperação

Prof. Dr. Edimar Manica

edimar.manica@ibiruba.ifrs.edu.br



Recuperação de Falhas

- O SGBD não deve permitir que algumas operações de uma transação T sejam aplicadas ao banco de dados enquanto outras operações de T não sejam.
 - Isso pode acontecer quando uma transação falha após executar algumas de suas operações (e não todas).



Tipos de falhas possíveis

- Falha no computador
 - Erro de transação ou de sistema
 - Imposição do controle de concorrência
 - Falha no disco
 - Problemas físicos e catástrofes
-
- Para os 3 primeiros tipos de falhas, o SGBD deve manter informações suficientes para se recuperar da falha.



Recuperação de Falhas - *Log* do Sistema

- Para manter a consistência do banco de dados, o gerenciador de recuperação registra no histórico (log), para cada transação, as operações que afetam os valores dos itens do banco:
 - **[start_transaction, T]**: indica que a transação T iniciou sua execução.
 - **[write_item, T, X, old_value, new_value]**: indica que a transação T alterou o valor do item X do banco de dados de old_value (valor antigo) para new_value (novo valor).
 - **[read_item, T, X]**: indica que a transação T leu o valor do item X do banco de dados.
 - **[commit, T]**: indica que a transação T foi finalizada com sucesso.
 - **[abort, T]**: indica que a transação T foi abortada.



Recuperação de Falhas - *Log* do Sistema

- Quando ocorre uma falha:
 - as transações inicializadas, mas que não gravaram seus registros de *commit* no *log*, devem ser desfeitas;
 - as transações que gravaram seus registros de *commit* no *log* podem ter que ser refeitas a partir dos registros do *log*.
- Para tanto, no processo de recuperação de falhas relativa a uma transação T, usam-se as operações:
 - **UNDO** (desfazer): desfaz a transação, ou seja, percorre o *log* de forma retroativa, retornando todos os itens alterados por uma operação *write* de T aos seus valores antigos.
 - **REDO** (refazer): refaz a transação, ou seja, percorre o *log* para frente, ajustando todos os itens alterados por uma operação *write* de T para seus valores novos.



Escalonamento e Recuperabilidade

- Quando as transações estão executando simultaneamente em um padrão intercalado, a ordem da execução das operações de todas as diversas transações é conhecida como um escalonamento (**schedule**)
- Um **escalonamento** S de n transações $T_1, T_2, \dots, T_n$ é uma ordenação das operações das transações
 - As operações das diferentes transações podem ser intercaladas no escalonamento S
 - Porém, para cada transação T_i que participa de S , as operações de T_i em S devem aparecer na mesma ordem em que ocorrem em T_i



Escalonamento e Recuperabilidade

- Notação simplificada para escalonamento:
 - $r_i(X)$: read_item(X) na transação T_i
 - $w_i(X)$: write_item(X) na transação T_i
 - c_i : commit na transação T_i
 - a_i : abort na transação T_i
- Exemplos de escalonamento
 - $S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y)$
 - $S_b: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); a_1$



Escalonamento e Recuperabilidade

- Duas operações em um escalonamento são ditas **conflitantes** se:
 - pertencem a diferentes transações
 - possuem acesso ao mesmo item X
 - pelo menos uma delas é uma operação `write_item(X)`



Escalonamento e Recuperabilidade

- Um escalonamento S é dito ser recuperável se nenhuma transação T em S entrar no estado confirmado até que todas as transações T' , que tenham escrito um item que T tenha lido, entrem no estado confirmado.
- $S_a: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; r_2(Y); c_2;$
 - S_a é recuperável porque T_1 é confirmada antes que T_2 (T_2 que lê um item atualizado por T_1)
- $S_c: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$
 - S_c não é recuperável, porque T_2 lê o item X atualizado por T_1 , e então T_2 é confirmado antes que T_1 se confirme.

Escalonamento e Recuperabilidade

- Em um escalonamento recuperável, pode ocorrer um fenômeno conhecido como rollback em cascata, no qual uma transação não-confirmada tenha que ser desfeita porque leu um item de uma transação que falhou.
 - $S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$
- Um escalonamento evita *rollbacks* em cascata se todas as transações no escalonamento lerem somente itens que tenham sido escritos por transações já confirmadas.
 - No escalonamento S_e anterior, $r_2(X)$ deve ser adiada até que T_1 tenha sido confirmada (ou abortada), retardando T_2 .



Escalonamento e Recuperabilidade

- Um escalonamento é denominado estrito se todas as suas transações não puderem ler nem escrever um item X até que a última transação que escreveu X tenha sido confirmada (ou abortada).



Seriabilidade de Escalonamentos

- Um escalonamento S é denominado **serial** se, para todas as transações T participantes do escalonamento, todas as operações de T forem executadas consecutivamente no escalonamento; caso contrário, o escalonamento é denominado **não-serial**.
- Um escalonamento **serial**:
 - possui somente uma transação ativa de cada vez
 - não permite nenhum entrelaçamento de transações
 - é considerado correto, independente da ordem de execução das transações
 - limita a concorrência
 - na prática, é inaceitável



Seriabilidade de Escalonamentos

- Um escalonamento S de n transações é serializável se for equivalente a algum escalonamento serial das n transações
 - Dizer que um escalonamento não-serial S é serializável equivale a dizer que ele é correto, já que equivale a um escalonamento serial que é considerado correto
 - Dois escalonamentos são ditos equivalentes se a ordem de quaisquer duas **operações conflitantes** for a mesma nos dois escalonamentos
- Exemplos
 - $S_a: r_1(X); w_1(X); r_1(Y); w_1(Y); c_1; r_2(X); w_2(X); c_2$; (serial)
 - $S_b: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y); c_1; c_2$; (serializável)
 - $S_c: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2$; (não serializável)



Teste de Seriabilidade

- Uma forma de testar a seriabilidade de um escalonamento é através da construção de um grafo de precedência.
- Um grafo de precedência de um escalonamento S é um grafo dirigido $G = (N, E)$, onde N é um conjunto de nodos $\{T_1, T_2, \dots, T_n\}$ e E é um conjunto de arcos dirigidos $\{e_1, e_2, \dots, e_m\}$ tal que:
 - cada nodo T_i corresponde a uma transação de S ;
 - cada arco e_j liga uma transação T_j que possui uma operação conflitante com uma transação T_k
- Um escalonamento S é serializável se e somente se o grafo de precedência não tiver nenhum ciclo.

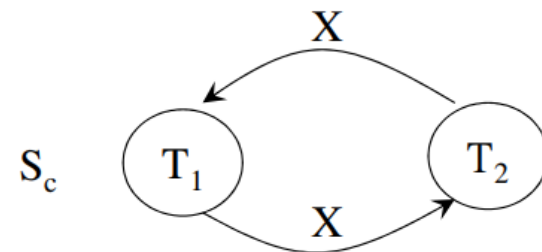
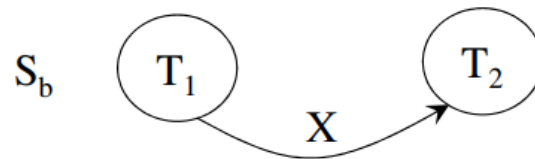
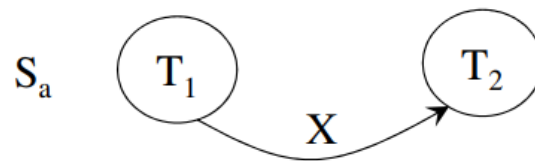
Teste de Seriabilidade

$S_a: r_1(X); w_1(X); r_1(Y); w_1(Y); c_1; r_2(X); w_2(X); c_2;$

$S_b: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y); c_1; c_2;$

$S_c: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$

- Para os escalonamentos S_a , S_b e S_c , os grafos de precedência são:



Principais Técnicas - Falhas não catastróficas

- Técnicas Principais para Recuperação de Falhas não catastróficas
 - Técnicas de atualização **adiada**
 - Técnicas de atualização **imediate**



Técnicas de Atualização Adiada

- Não atualizam fisicamente o banco de dados no disco até que uma transação atinja seu ponto de confirmação, então as atualizações são registradas no banco de dados
- Antes de atingir a confirmação, todas as atualizações da transação são registradas no espaço de trabalho de transação local ou nos buffers da memória principal que o SGBD mantém (o cache de memória principal do SGBD)
- Antes da confirmação, as atualizações são gravadas persistentemente no log e, após a confirmação, elas são gravadas no banco de dados no disco, a partir dos buffers na memória principal
- Se uma transação falhar antes de atingir seu ponto de confirmação, ela não terá alterado o banco de dados de forma alguma, de modo que o UNDO não é necessário



Técnicas de Atualização Imediata

- O banco de dados pode ser atualizado por algumas operações de uma transação antes que a transação alcance seu ponto de confirmação
- Porém, essas operações também precisam ser registradas no *log* no disco ao forçar a gravação antes que elas sejam aplicadas ao banco de dados no disco, tornando a recuperação ainda possível
- Se uma transação falhar depois de gravar algumas mudanças no disco, mas antes de atingir seu ponto de confirmação, o efeito de suas operações no banco de dados precisa ser desfeito; ou seja, a transação deve ser revertida

